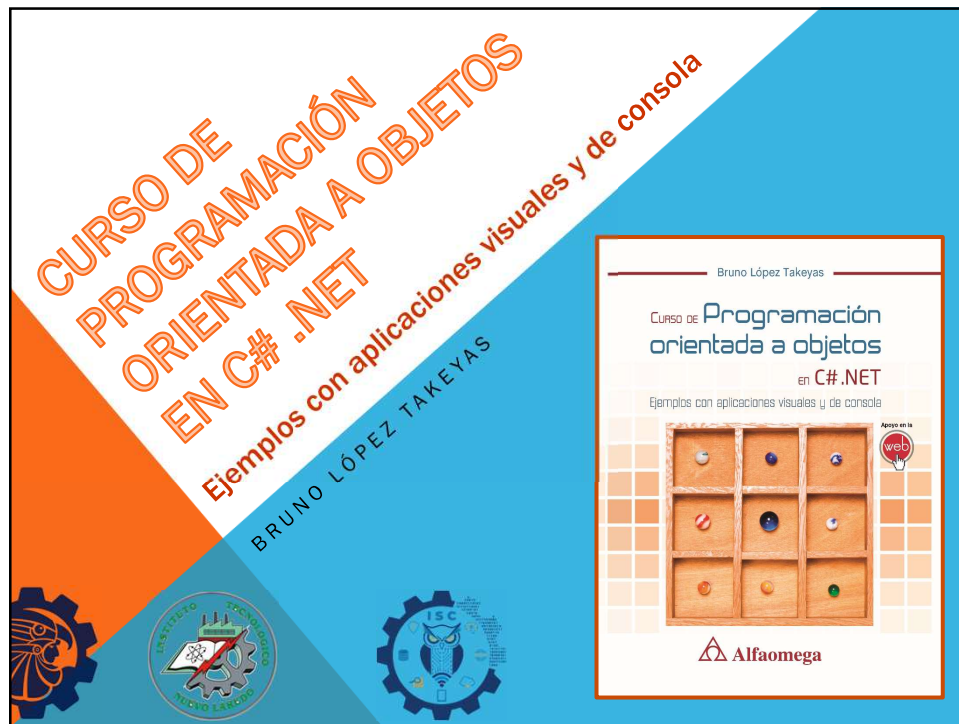




CAPITULO 8

MÉTODOS DE ORDENAMIENTO CON DELEGADOS

3	1	4	1	5	0	2	6	5	4	$i = 10$
1	3	1	4	5	2	6	5	4	0	$i = 9$
1	1	3	4	2	5	4	6	0		$i = 8$
1	1	3	2	4	5	4	5	6	0	$i = 7$
1	1	2	3	4	4	5	5	6	0	$i = 6$
1	1	2	3	4	4	5	5	6	0	$i = 5$
1	1	2	3	4	4	5	5	6	0	$i = 4$
1	1	2	3	4	4	5	5	6	0	$i = 3$
1	1	2	3	4	4	5	5	6	0	$i = 2$

Métodos de ordenamiento

- Introducción
- Criterios de ordenamiento
- Tipos de ordenamiento
- Métodos de ordenamiento interno

Objetivo

- Conocer un algoritmo para ordenar datos almacenados en un arreglo de tal forma que puedan ser fácilmente procesados en aplicaciones para resolver problemas cotidianos.

Ordenamiento de los datos de un arreglo

- Los algoritmos que ordenan datos de los arreglos se conocen como “**sorteadores**” o “**métodos de ordenamiento**”
- **Existen muchos métodos de ordenamiento**
- Cada método de ordenamiento tiene una estrategia para reacomodar sus datos
- **No se “mueven” las celdas del arreglo, sino su contenido**

4

Criterios de ordenamiento

- **Ascendente:** Los datos se acomodan en secuencia del menor al mayor, es decir, cada dato sucesor debe ser mayor ó igual que su antecesor. En este caso $DATO_0 \leq DATO_1 \leq DATO_2 \leq \dots \leq DATO_n$.
- **Descendente:** Los datos se colocan sucesivamente del mayor al menor, o sea, cada dato sucesor debe ser menor ó igual que su antecesor. En este caso $DATO_0 \geq DATO_1 \geq DATO_2 \geq \dots \geq DATO_n$.

Método de la burbuja

- Es el método de ordenamiento de datos más sencillo de implementar
- Su nombre se debe a que durante el ordenamiento de los datos el elemento a ordenar “burbujea” hacia arriba en el arreglo
- Existen varias versiones:
 - *Burbuja izquierda*
 - *Burbuja derecha*
 - *Burbuja con señal*

Principio del funcionamiento de la burbuja izquierda

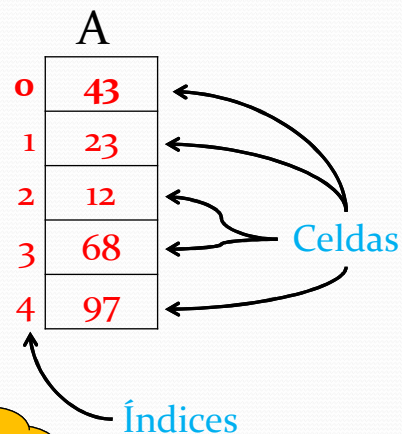
- Recorre el arreglo de derecha a izquierda
- Compara los datos adyacentes en el arreglo
- Hace los intercambios pertinentes
- Implementa un ciclo que controla la posición que ocupará el dato del arreglo (i)
- Dentro de este ciclo hay otro ciclo que recorre el arreglo de derecha a izquierda para hacer las comparaciones (j)
- Los desplazamientos de los datos se hacen hacia la izquierda del arreglo

Arreglo

Conjunto homogéneo
y estático
de datos relacionados
e indexados

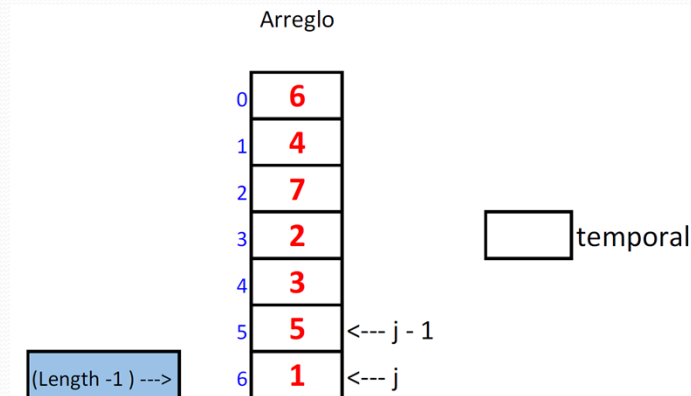
Homogéneo significa que todas sus celdas son del mismo tipo de dato

Estático se refiere a que, una vez declarado, no cambia su tamaño



Intercambio de datos

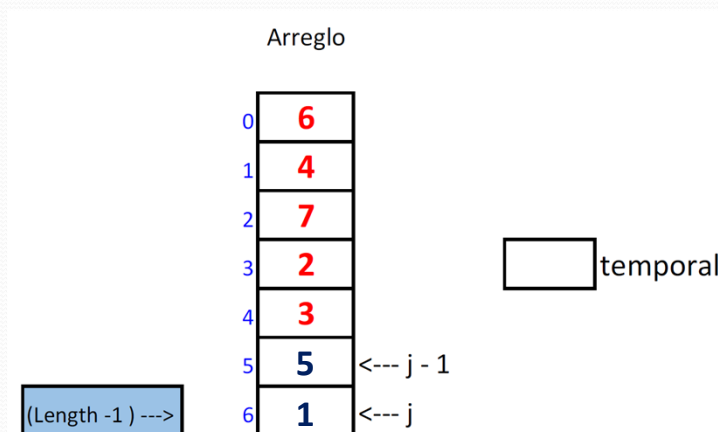
- *Algunos métodos de ordenamiento requieren intercambiar datos*
- *Para lograrlo se apoyan en una variable auxiliar (temporal)*



9

Ejemplo de intercambio de datos

- 1) *Copiar el valor del primer dato en la variable temporal.*
- 2) *Copiar el valor del segundo dato en la variable del primer dato.*
- 3) *Copiar el valor de la variable temporal en la variable del segundo dato.*



10

Método para intercambiar datos

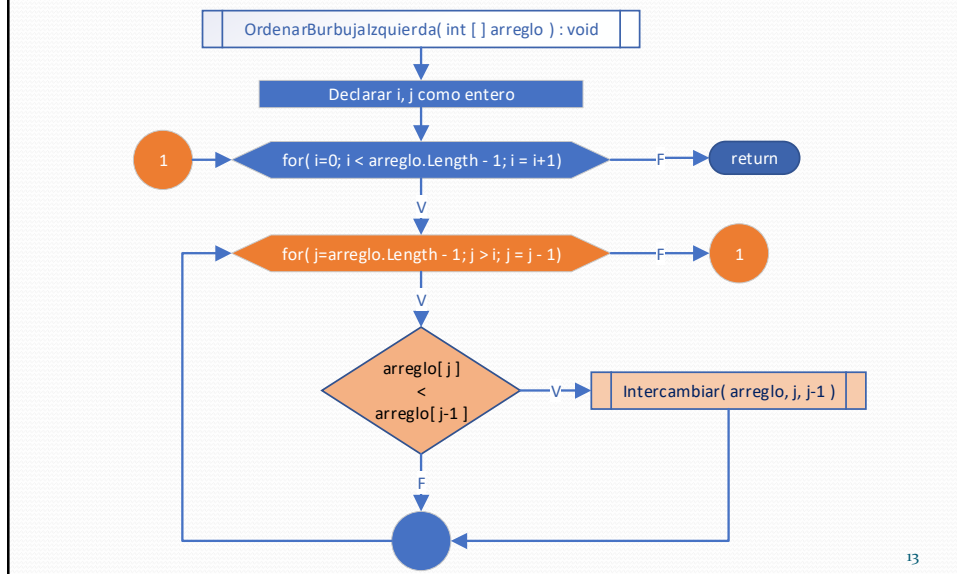
```
Intercambiar(arreglo[], j, j-1): nulo  
1.-temporal = arreglo[j-1]  
2.-arreglo[j-1] = arreglo[j]  
3.-arreglo[j] = temporal  
{Fin del método}
```

11

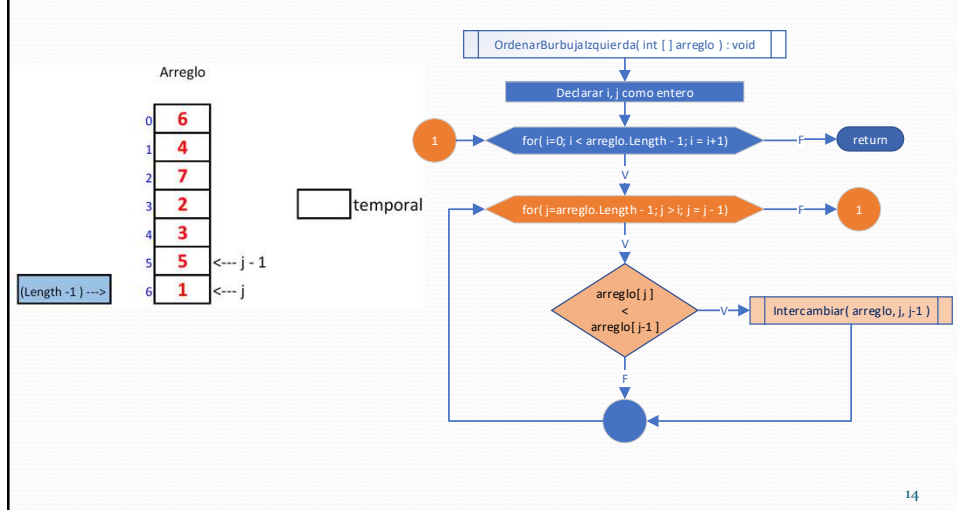
Pseudocódigo de la burbuja izquierda

```
OrdenarBurbujaIzquierda(arreglo[]): nulo  
  
1.- REPETIR CON i DESDE 0 HASTA arreglo.Length-1 CON PASO 1  
    1.1. REPETIR CON j DESDE arreglo.Length-1 HASTA i CON PASO -1  
        1.1.1 SI arreglo[j] < arreglo[j-1] ENTONCES  
            1.1.1.1 Intercambiar(arreglo, j, j-1)  
        1.1.2. {FIN DE LA CONDICIONAL DEL PASO 1.1.1}  
    1.2. {FIN DEL CICLO DEL PASO 1.1}  
  
2.- {FIN DEL CICLO DEL PASO 1}  
3.- RETURN
```

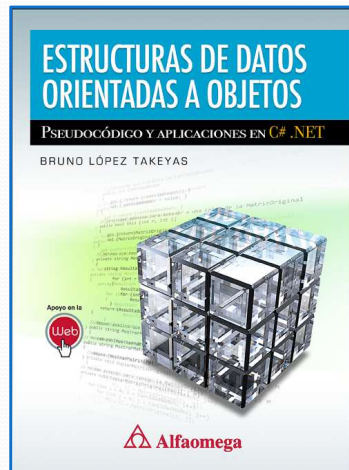
Diagrama de flujo del método de la burbuja izquierda que ordena números enteros



Funcionamiento del método de la burbuja



LECTURA



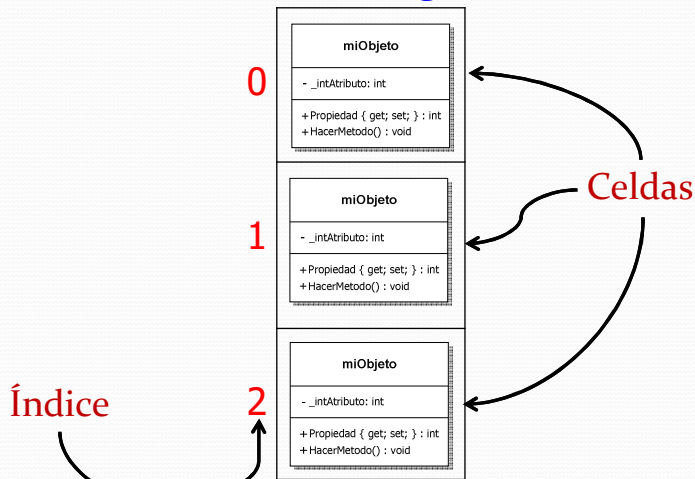
Para consultar este tema con mayor profundidad y revisar otros métodos de ordenamiento se recomienda la lectura de:

Capítulo 10.- Métodos de ordenamiento

15

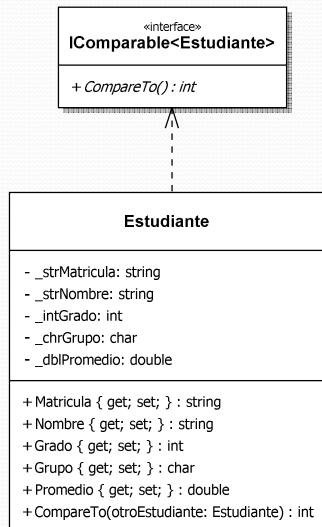
Arreglo de objetos

miArreglo



16

Declaración y creación de un arreglo de objetos



- Suponga que desea un arreglo que almacene los estudiantes de una escuela

- Datos

- *Matrícula*
- *Nombre*
- *Grado*
- *Grupo*
- *Promedio*

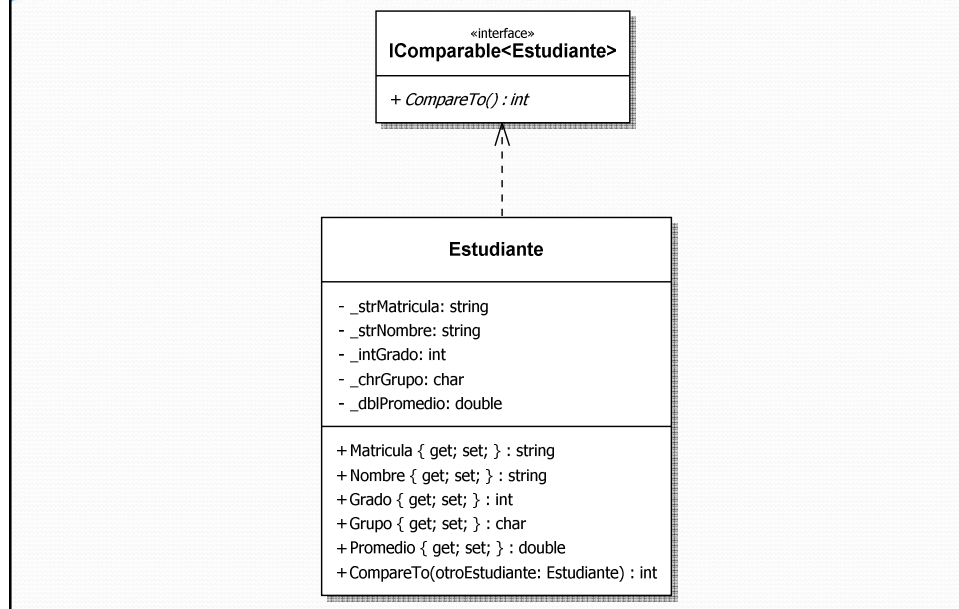
17

Diseño de una aplicación sencilla (sin delegados)

- Ordenar datos de estudiantes
- Datos de los estudiantes
 - *Matrícula (string)*
 - *Nombre (string)*
 - *Grado (int)*
 - *Grupo (char)*
 - *Promedio (double)*
- Ordenar los objetos de manera ascendente por promedio

18

Definición de la clase con los datos



Arreglo de objetos de tipo Estudiante

```
const int intTamaño = 10;

Estudiante [] miArreglo;
miArreglo = new Estudiante[intTamaño];
```

Diagrama de flujo del método de comparación CompareTo()

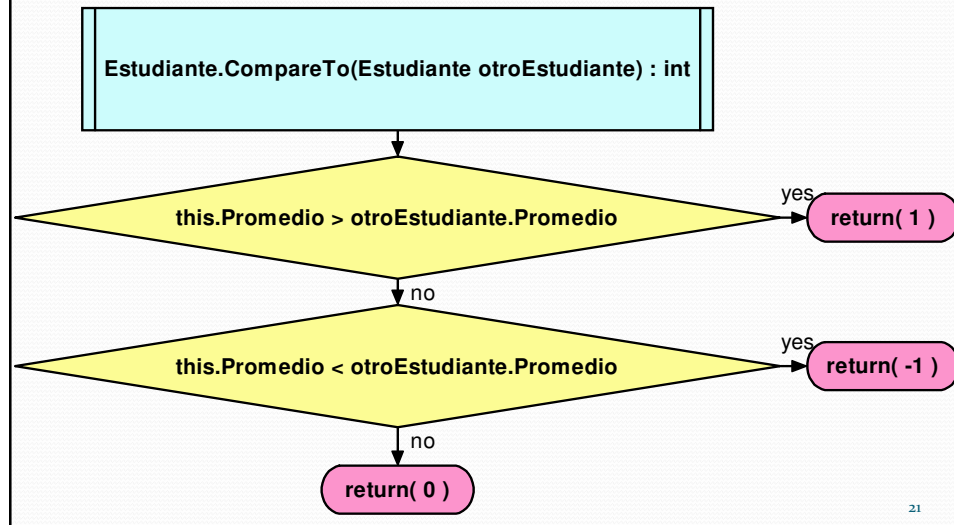
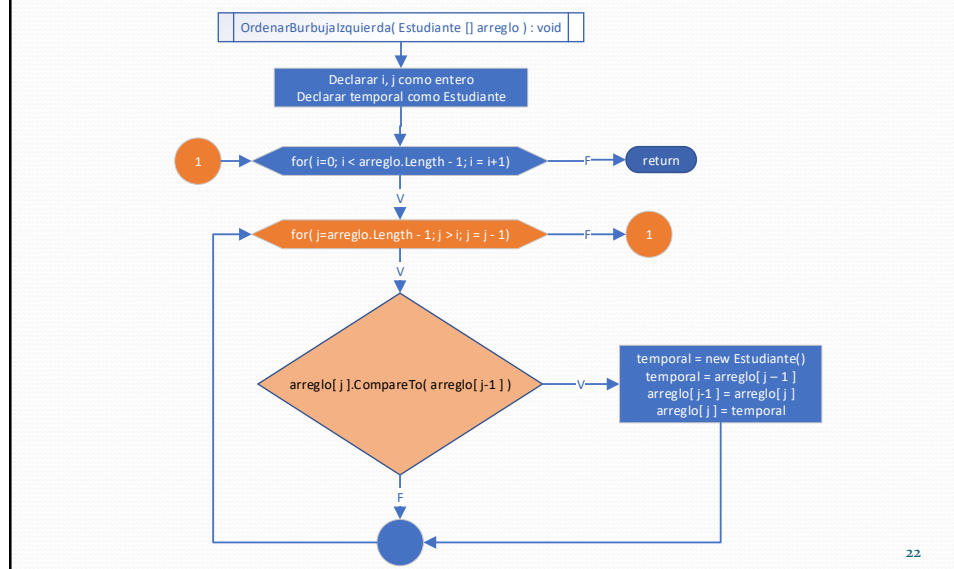


Diagrama de flujo del método para ordenar un arreglo de objetos de tipo Estudiante



Diseño de la forma

	Matrícula	Nombre	Grado	Grupo	Promedio
▶	5	Bruno	6	A	94.4
	4	Pepe	2	C	88.3
	6	Fely	5	B	93.2
	27	Pancho	3	B	90.1
	23	Raúl	4	D	92.2
	3	Ricky	2	A	89.4
	7	Budy	7	A	95.6

23

Características de la aplicación

- Ordena datos de estudiantes
 - Criterio de ordenamiento: Ascendente
 - Considera el campo del **promedio** para hacer las comparaciones de los objetos
- ¿Cómo seleccionar el criterio de ordenamiento?
Ascendente o descendente
 - ¿Cómo seleccionar el campo a utilizar para comparar los objetos?
 - *Matrícula, Nombre, Grado, Grupo o Promedio*

RECORDATORIO: DELEGADOS

- Es un nuevo **tipo** que hace referencia a un método
- Permiten enviar los métodos como parámetros

Sintaxis

```
modificador delegate tipo nombre(parámetros);
```

Modificadores:

- `private`
- `protected`
- `public`
- `internal`

¿Para qué sirve un delegado?

- Si un delegado es un tipo, entonces se pueden declarar variables de dicho tipo.
- Un delegado es una referencia a un método.
- Una variable creada de un tipo delegado representa a un método determinado.



Los delegados se utilizan para enviar métodos como parámetros a otros métodos.



Declaración de una variable de un delegado

```
Delegado w;
```

donde

- **Delegado** representa un tipo que hace referencia a un método
- “w” es una variable que almacena sentencias

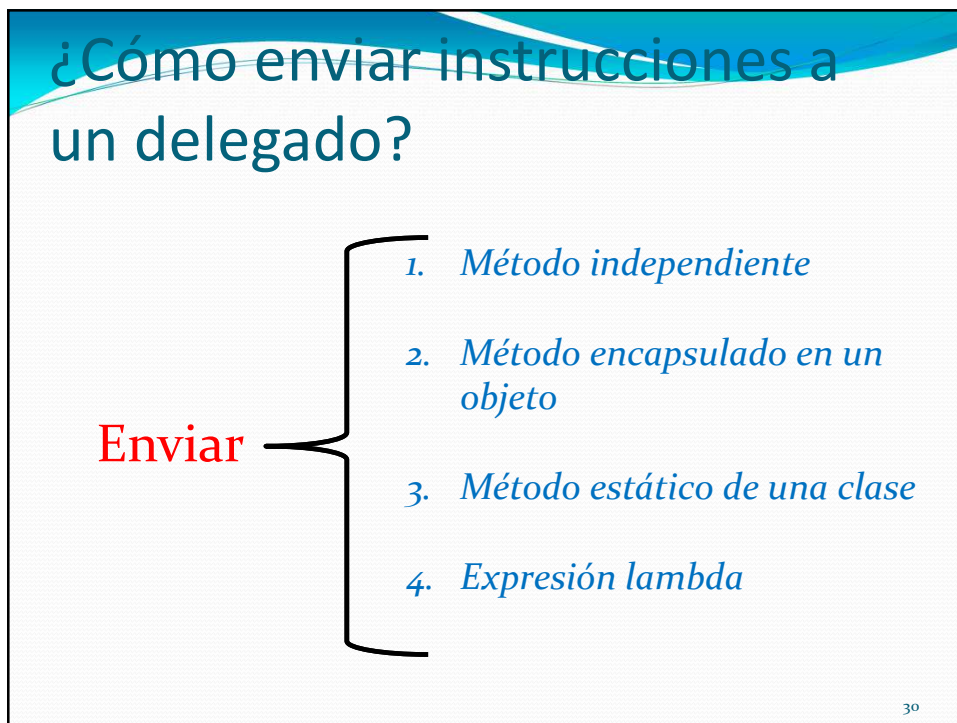
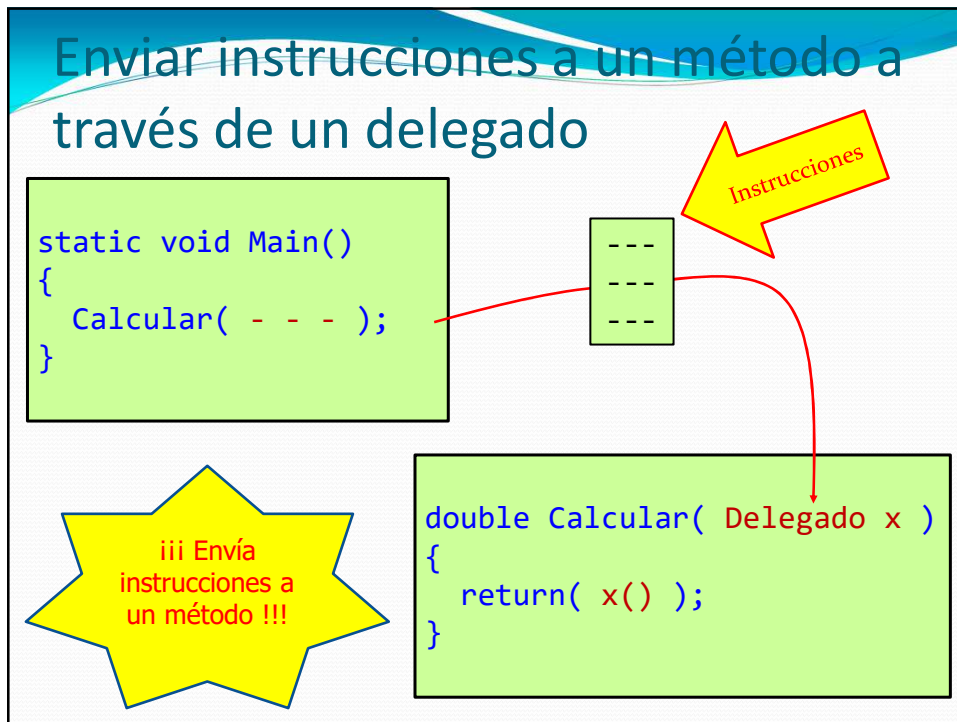
Contenido de
la variable w =

```
Console.Clear();  
Console.Write("Hola mundo");
```

Ejemplo de delegados

```
public delegate double Delegado(double x, double y);
```

- Cualquier método puede asignarse a un delegado.
- Deben coincidir...
 - *El prototipo del delegado*
 - *Parámetros*
 - *Tipo de dato del valor devuelto*



1.- Enviar un método independiente a un delegado

```
static void Main()  
{  
    Calcular( Hacer() );  
}
```

return (3.0 * 2.5);

```
double Hacer()  
{  
    return ( 3.0 * 2.5 );  
}
```

```
double Calcular( Delegado x )  
{  
    return( x() );  
}
```

Instrucciones

2.- Enviar un método encapsulado en un objeto a un delegado

```
static void Main()  
{  
    Calcular( unObjeto.Calcular() );  
}
```

unObjeto.Calcular()

unObjeto

+ Calcular() : double

```
double Calcular( Delegado x )  
{  
    return( x() );  
}
```

Instrucciones

3.- Enviar un método estático de una clase a un delegado

```
static void Main()  
{  
    Calcular( Clase.Calcular() );  
}
```

Clase.Calcular()

Instrucciones

Clase

+ Calcular() : double

static

```
double Calcular( Delegado x )  
{  
    return( x() );  
}
```

4.- Enviar una expresión lambda a un delegado

```
static void Main()  
{  
    Calcular( ()=>Math.PI * 6.5 );  
}
```

() => Math.PI * 6.5

Instrucciones

```
double Calcular( Delegado x )  
{  
    return( x() );  
}
```

Diseño de una aplicación con delegados

- Ordenar datos de estudiantes
- Seleccionar el criterio de ordenamiento
 - Ascendente
 - Descendente
- Seleccionar el campo para hacer las comparaciones
 - Matrícula (string)
 - Nombre (string)
 - Grado (int)
 - Grupo (char)
 - Promedio (double)

35

Definición de la clase con los datos

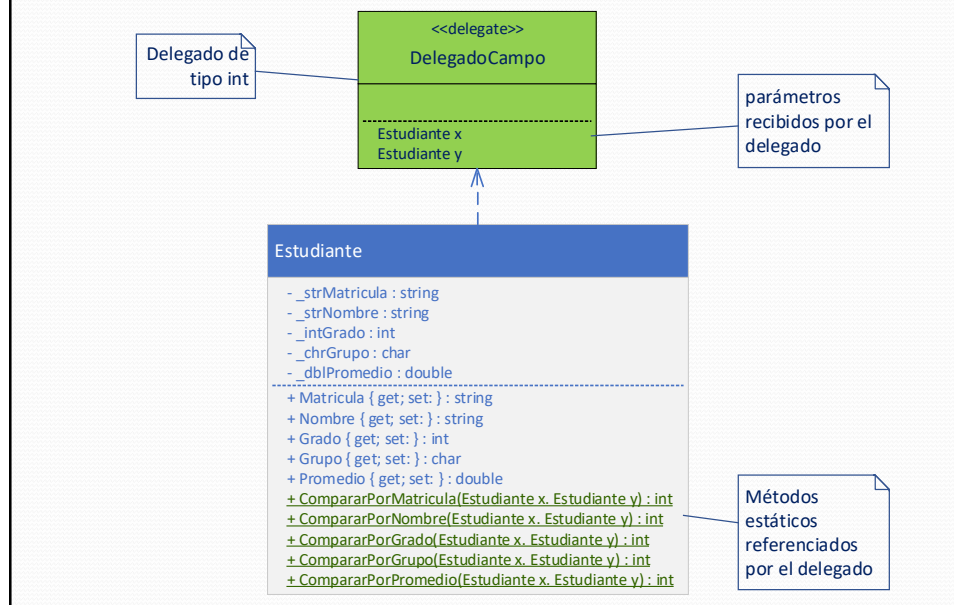


Diagrama de flujo del método de comparación para ordenar por matrícula

Estudiante.CompararPorMatricula(Estudiante x, Estudiante y) : int

return(x.Matricula.CompareTo(y.Matricula))

37

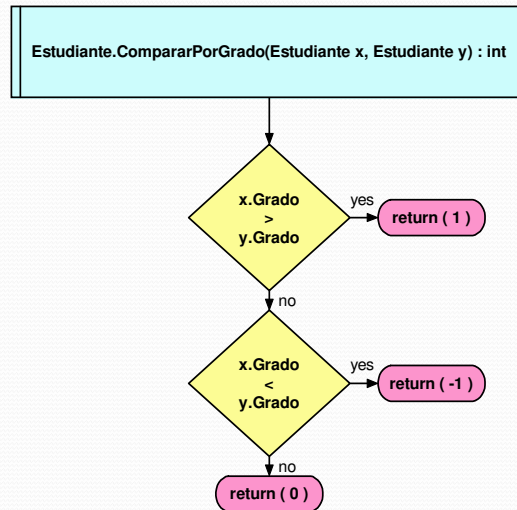
Diagrama de flujo del método de comparación para ordenar por nombre

Estudiante.CompararPorNombre(Estudiante x, Estudiante y) : int

return(x.Nombre.CompareTo(y.Nombre))

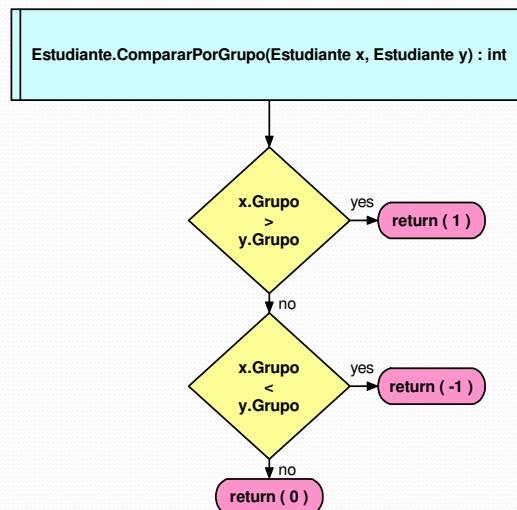
38

Diagrama de flujo del método de comparación para ordenar por grado



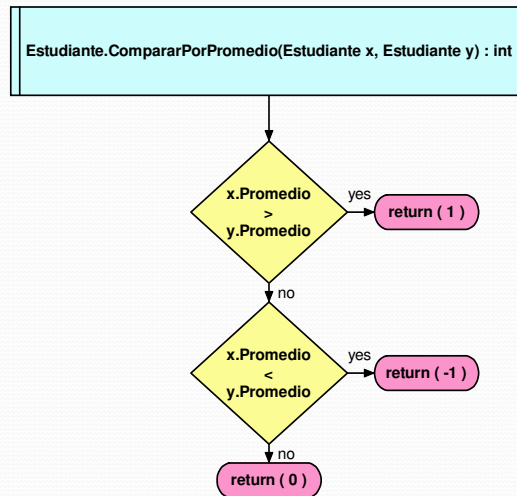
39

Diagrama de flujo del método de comparación para ordenar por grupo



40

Diagrama de flujo del método de comparación para ordenar por promedio

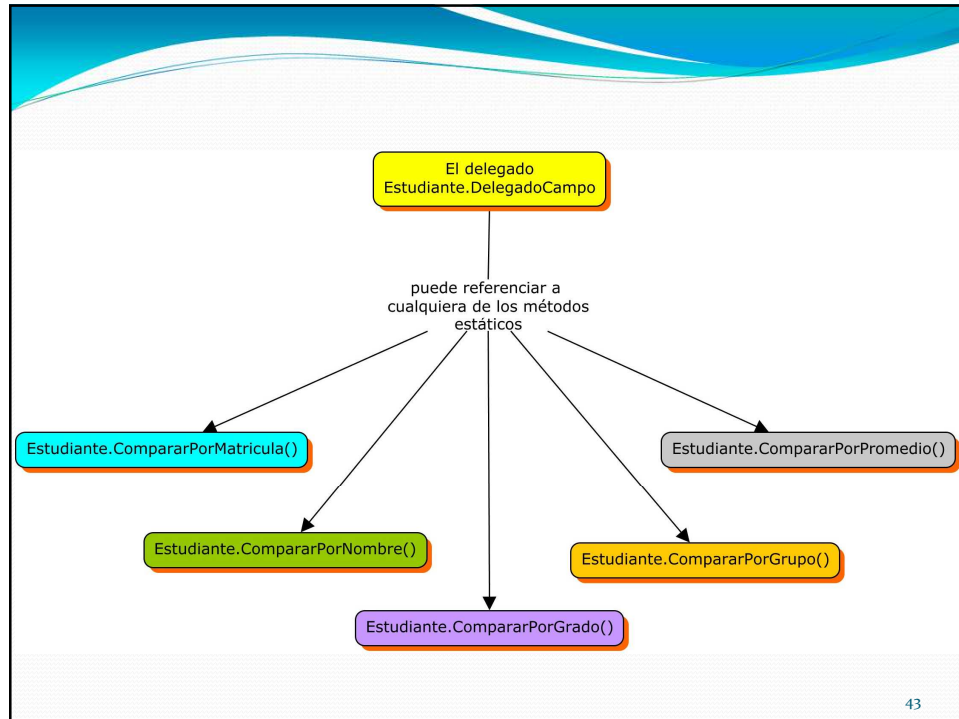


41

El delegado DelegadoCampo

- Permite declarar una variable como:
 - `Estudiante.DelegadoCampo campo;`
- La variable `campo` representa un método que puede contener la referencia a cualquiera de los siguientes métodos estáticos de la clase `Estudiante`:
 - `Estudiante.CompararPorMatricula(Estudiante x, Estudiante y)`
 - `Estudiante.CompararPorNombre(Estudiante x, Estudiante y)`
 - `Estudiante.CompararPorGrado(Estudiante x, Estudiante y)`
 - `Estudiante.CompararPorGrupo(Estudiante x, Estudiante y)`
 - `Estudiante.CompararPorPromedio(Estudiante x, Estudiante y)`

42



El delegado `DelegadoCampo(cont.)`

- Recibe **2** parámetros que representan los objetos de los estudiantes que se desean comparar:
 - `Estudiante x`
 - `Estudiante y`
- Esos objetos están ubicados en el arreglo en las celdas **j** y **j-1** respectivamente.

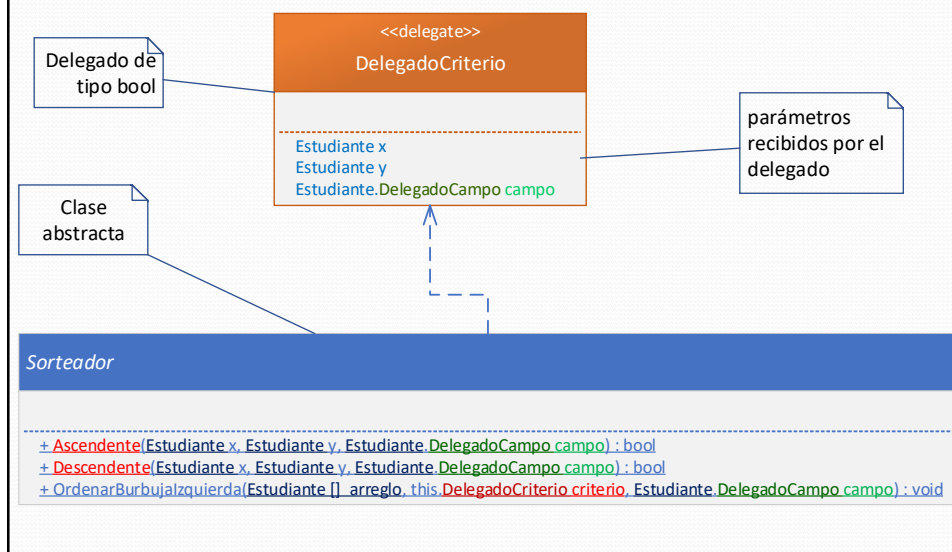
44

El delegado DelegradoCampo (cont.)

- Devuelve un valor de tipo entero:
 - **1** cuando el *Estudiante x* es mayor que el *Estudiante y*
 - **-1** cuando el *Estudiante x* es menor que el *Estudiante y*
 - **0** cuando el *Estudiante x* es igual que el *Estudiante y*

45

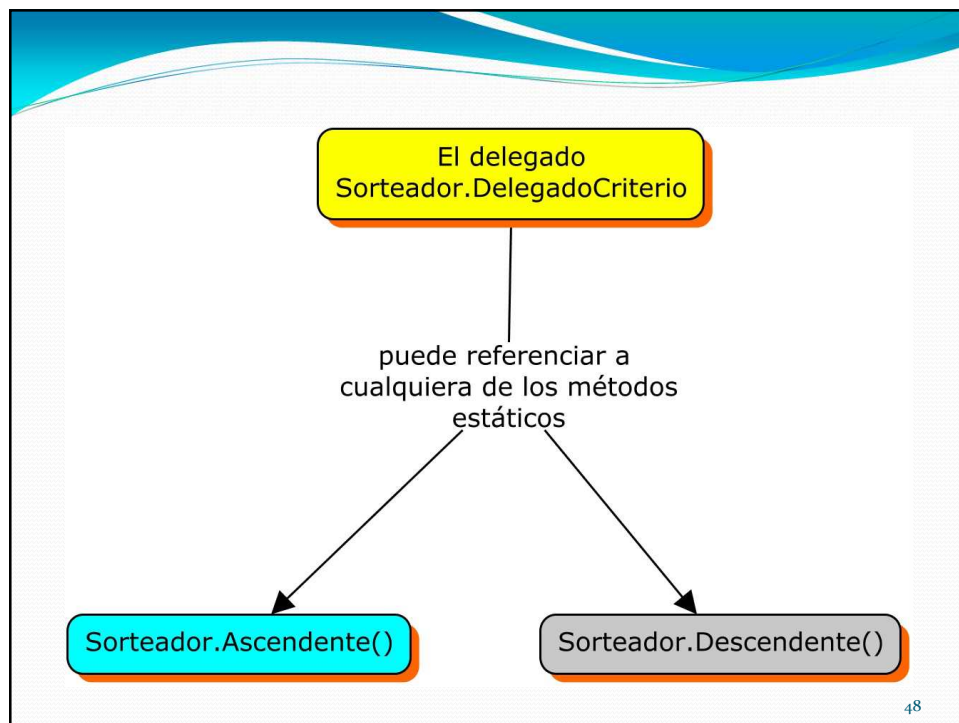
Definición de la clase con el método de ordenamiento



El delegado **DelegadoCriterio**

- Permite declarar una variable como:
 - `Sorteador.DelegadoCriterio criterio;`
- La variable `criterio` representa un método que puede contener la referencia a cualquiera de los siguientes métodos estáticos de la clase `Sorteador`:
 - `Sorteador.Ascendente()`
 - `Sorteador.Descendente()`

47



48

El delegado **DelegadoCriterio** (cont.)

- Recibe 3 parámetros que representan los objetos de los estudiantes que se desean comparar y el método de comparación:
 - Estudiante x
 - Estudiante y
 - Estudiante.DelegadoCampo campo
- Esos objetos están ubicados en el arreglo en las celdas j y j-1 respectivamente.

49

El delegado **DelegadoCriterio** (cont.)

- Devuelve un valor de tipo booleano:
 - true cuando se cumple la condición de la comparación y se intercambiarán los objetos Estudiante x y Estudiante y en el arreglo
 - false en caso contrario

50

Diagramas de clases

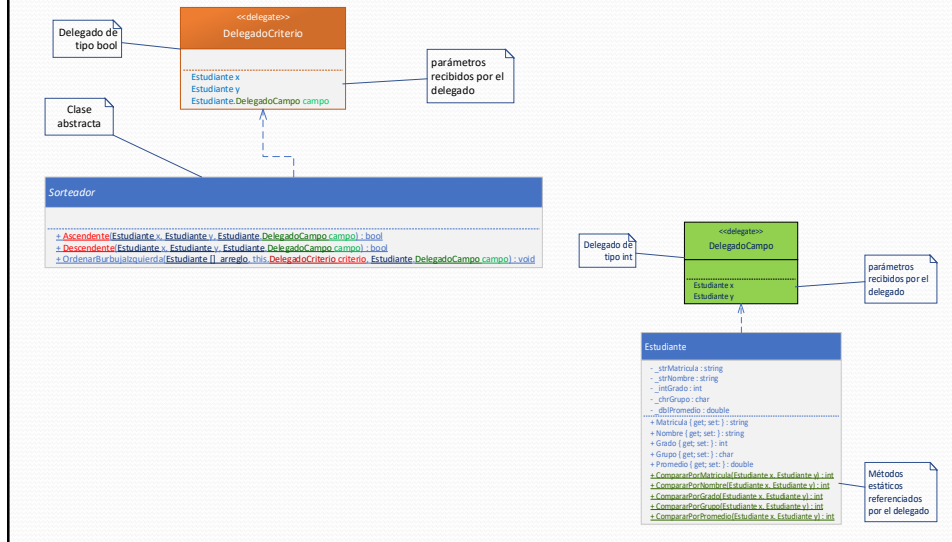
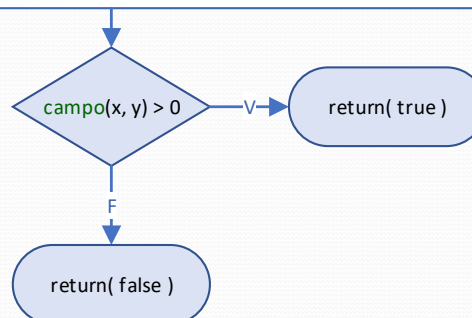


Diagrama de flujo del método para el criterio de ordenamiento ascendente

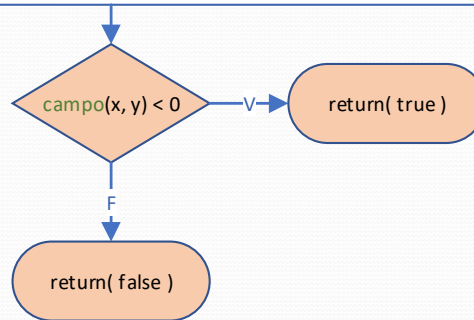
`Sorteador.Ascendente(Estudiante x, Estudiante y, Estudiante.DelegadoCampo campo) : bool`



52

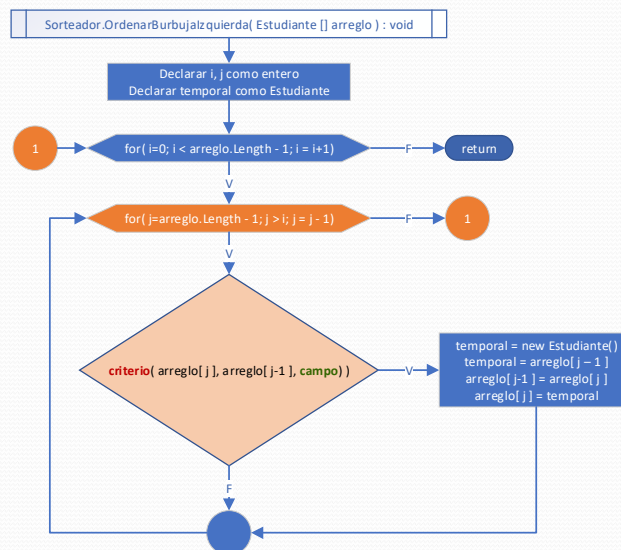
Diagrama de flujo del método para el criterio de ordenamiento descendente

```
Sorteador.Descendente(Estudiente x, Estudiante y, Estudiante.DelegadoCampo campo) : bool
```



53

Diagrama de flujo del método de ordenamiento con delegados



54

Diseño de la forma

	Matrícula	Nombre	Grado	Grupo	Promedio
▶	5	Bruno	6	A	94.4
	4	Pepe	2	C	88.3
	6	Fely	5	B	93.2
	27	Pancho	3	B	90.1
	23	Raúl	4	D	92.2
	3	Ricky	2	A	89.4
	7	Budy	7	A	95.6

55

Codificación

```

if (radAscendente.Checked && radPorMatricula.Checked)
    Sorteador.OrdenarBurbujaIzquierda(arreglo, Sorteador.Ascendente, Estudiante.CompararPorMatricula);

if (radAscendente.Checked && radPorNombre.Checked)
    Sorteador.OrdenarBurbujaIzquierda(arreglo, Sorteador.Ascendente, Estudiante.CompararPorNombre);

if (radAscendente.Checked && radPorGrado.Checked)
    Sorteador.OrdenarBurbujaIzquierda(arreglo, Sorteador.Ascendente, Estudiante.CompararPorGrado);

if (radAscendente.Checked && radPorGrupo.Checked)
    Sorteador.OrdenarBurbujaIzquierda(arreglo, Sorteador.Ascendente, Estudiante.CompararPorGrupo);

if (radAscendente.Checked && radPorPromedio.Checked)
    Sorteador.OrdenarBurbujaIzquierda(arreglo, Sorteador.Ascendente, Estudiante.CompararPorPromedio);

...
    
```

56

Codificación (cont.)

```
if (radDescendente.Checked && radPorMatricula.Checked)
    Sorteador.OrdenarBurbujaIzquierda(arreglo, Sorteador.Descendente, Estudiante.CompararPorMatricula);

if (radDescendente.Checked && radPorNombre.Checked)
    Sorteador.OrdenarBurbujaIzquierda(arreglo, Sorteador.Descendente, Estudiante.CompararPorNombre);

if (radDescendente.Checked && radPorGrado.Checked)
    Sorteador.OrdenarBurbujaIzquierda(arreglo, Sorteador.Descendente, Estudiante.CompararPorGrado);

if (radDescendente.Checked && radPorGrupo.Checked)
    Sorteador.OrdenarBurbujaIzquierda(arreglo, Sorteador.Descendente, Estudiante.CompararPorGrupo);

if (radDescendente.Checked && radPorPromedio.Checked)
    Sorteador.OrdenarBurbujaIzquierda(arreglo, Sorteador.Descendente, Estudiante.CompararPorPromedio);
```

57

LECTURA

Para generar datos aleatorios, se recomienda la lectura de los apuntes



¿Cómo generar datos aleatorios en C#?

Incluye generar:

- Nombres aleatorios
- Sexo
- Fecha
- Datos booleanos
- etc.

<https://nlaredo.tecnm.mx/takeyas/Apuntes/Fundamentos%20de%20Programacion/Apuntes/07.-Aleatorios.pdf>

58

¿Cómo cargar datos preliminares?

```
Estudiante miEstudiante; // Declaración del objeto
Random aleatorio = new Random(); // Declaración del número aleatorio
for (int i = 0; i < 1000; i++) // Genera 1000 objetos con datos aleatorios
{
    miEstudiante = new Estudiante(); // Crea el objeto miEstudiante

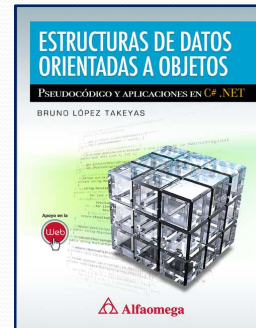
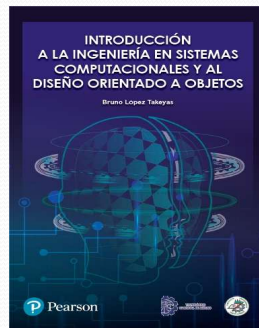
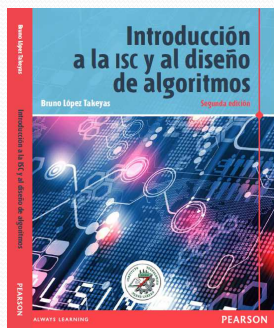
    // Genera de manera aleatoria los datos de miEstudiante
    miEstudiante.Matricula = Guid.NewGuid().ToString().Substring(0, 9);
    miEstudiante.Nombre = Guid.NewGuid().ToString().Substring(0, 25);
    miEstudiante.Grado = aleatorio.Next(0, 14);
    miEstudiante.Grupo = char.Parse(Guid.NewGuid().ToString().Substring(0, 1));
    miEstudiante.Promedio = aleatorio.Next(0, 100);

    // Agrega los datos de miEstudiante al dataGridView dgEstudiantes
    dgEstudiantes.Rows.Add(miEstudiante.Matricula, miEstudiante.Nombre,
    miEstudiante.Grado.ToString(), miEstudiante.Grupo.ToString(),
    miEstudiante.Promedio.ToString("N2"));
}
```

59

Otros títulos del autor

<https://nlaredo.tecnm.mx/takeyas/Libro>



 bruno.lt@nlaredo.tecnm.mx

 Bruno López Takeyas